

TP2

Utilisation du langage assembleur sur ARM Cortex M0+

Dubresson Matteo

– 26 septembre 2026 –

Table des matières

1	Préparation	2
2	Manipulation 1	3
3	Manipulation 2	3
4	Manipulation 3	3
5	Manipulation 4	3

1 Préparation

P.1

Dans la documentation du LPC804, on peut retrouver les adresses de ces différents registres.

`SYSAHBCLKCTRL0 : 0x4004 8080 | 0100 0000 0000 0000 0100 1000 0000 1000 0000`

`DIR0 : 0xA000 2000 | 1010 0000 0000 0000 0010 0000 0000 0000`

P.2

Dans la documentation des processeurs ARMv6-M, on retrouve page A1-26 que les instructions sont codées sur 32 bits.

P.3

La ligne 10 du code active l'horloge des périphériques GPIO et IOCON. On réalise un `|=` pour ne pas changer les périphériques déjà activés. On combine également les deux périphériques avec un `|` pour pouvoir réaliser l'opération en une seule fois.

P.4

Dans le code assembleur présenté, on retrouve plusieurs instructions :

- `movs r3,#x` : permet de mettre une valeur `x` dans un registre `r3`.
- `lsls r2,r3,#y` : permet de décaler les bits du registre `r3` vers la gauche de `y` positions et la stocker dans `r2`.
- `ldr r1,[r0,r2]` : permet de lire la valeur à l'adresse contenue dans `r0 + r2` et de la stocker dans `r1`.
- `orrs r1,r3` : fait `r1 |= r3`.
- `str r1,[r0,r2]` : permet d'écrire la valeur contenue dans `r1` à l'adresse contenue dans `r0 + r2`.

P.5

Le `#` sert à désigner une valeur immédiate et non un registre.

Le `[]` sert à désigner une adresse mémoire.

P.6

On commence par mettre `0xA0` dans `R3`, on le décale de 24 bits puis on le met dans `R2`. On réalise la même opération avec `0x80` mais on le décale de 6 bits. En sommant les deux dans `R3`, on obtient `0xA000 2000` qui est l'adresse du registre `DIR0`.

On peut donc récupérer la valeur du registre `DIR0`, la suite du code servant à d'autres opérations.

2 Manipulation 1

1.1

1.2

Lorsqu'on implémente le code dans le microcontrôleur, on peut observer que la LED s'allume et s'éteint à une période de 0,57 s

1.3

Lorsqu'on lance le code dans l'onglet désassembleur, on constate que 48 instructions sont nécessaires pour exécuter la fonction main.

3 Manipulation 2

2.1

Les directives qui ne génèrent pas de code sont :

- .syntax
- .equ
- boucle / bne boucle

Tandis que les instructions qui génèrent du code sont :

- ldr
- str
- subs

2.2

Les instructions ont pour tâche

- ldr R,#x : permet de charger la valeur x dans le registre R.
- str R1,[R2,#0] : permet d'écrire la valeur contenue dans R1 à l'adresse contenue dans R2.
- subs R6,#1 : fait R6-=1

4 Manipulation 3

5 Manipulation 4